YuviPep AI Sales Platform — Backend

Architecture and capability reference for engineering and management

> *Everything below was read directly from the source in `app/` and verified by running the code. Nothing is taken from design documentation on trust.*

---

1. What this backend does

It runs the working life of a services business: a field representative meets a client, and from that conversation the system produces a project, a dated plan, staffed tasks, and every message that goes back to the client — while a human keeps every decision that matters.

The design intent is *zero-touch operations with human authority*. The AI does the clerical work — listening, drafting, scheduling, chasing, remembering — and stops at the point where judgement is required, where it hands to a named person.

It is **multi-tenant**: several client companies run on one deployment, fenced from each other at the application layer and, in production, at the database layer.

Technology. FastAPI · SQLAlchemy 2.0 (async) · SQLite for development, PostgreSQL for production · PyJWT · Fernet · OpenAI Whisper for on-premise speech · a pluggable LLM layer supporting Gemini, Claude and OpenRouter. No message broker, no Redis, no Celery — the queue and scheduler run in-process.

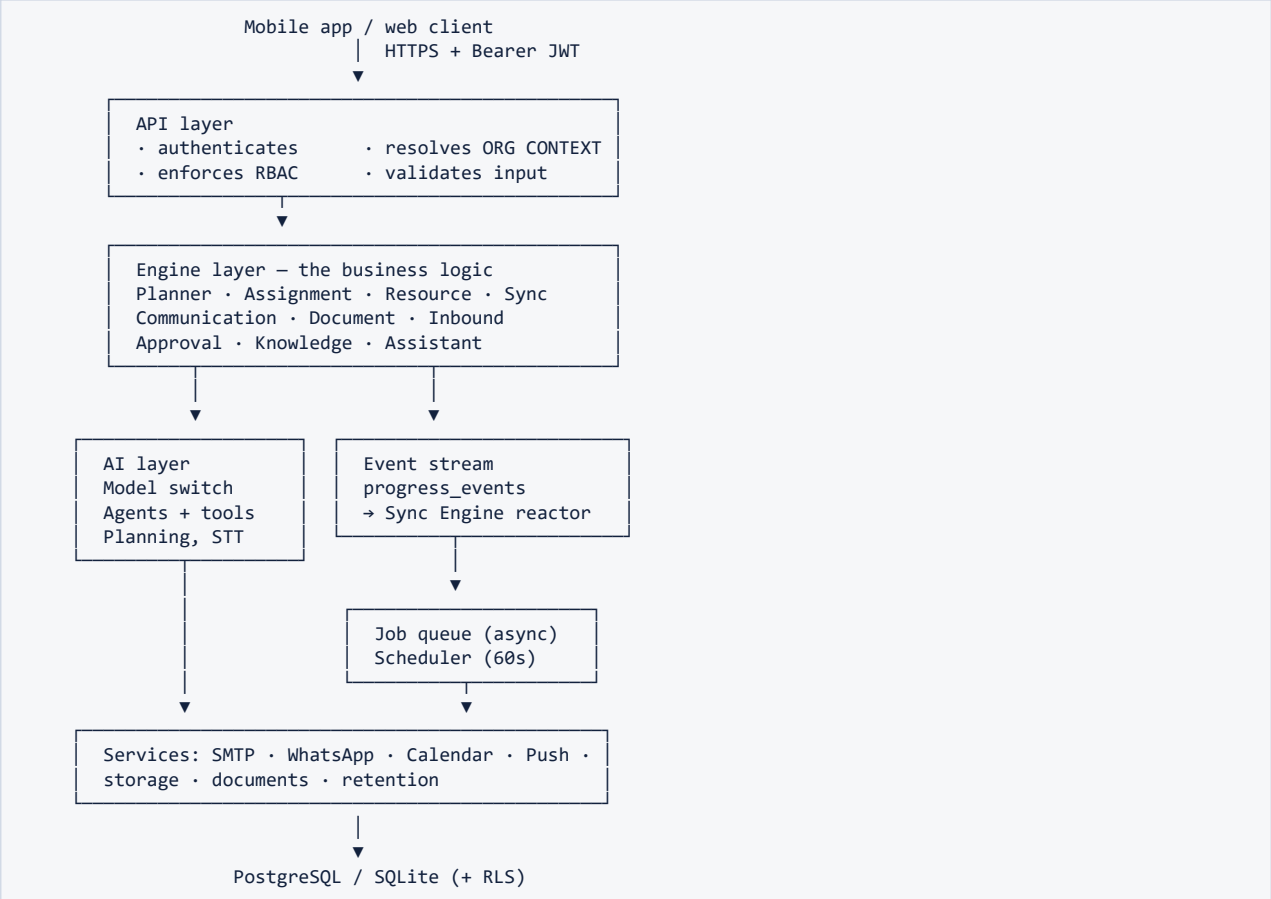
By the numbers

Every figure here was measured from the code, not estimated. They are stated so a reviewer can re-derive them and catch this document drifting. **Re-measured 2026-08-25** — the line and test counts had drifted; routes, tables, engines, permissions and plan operations had not, which is what `scripts/check_docs_in_sync.py` asserts on every run.

	Count	Where
Python in `app/`	30,576 lines	`app/`
Database tables	**55**	55 model classes, 755 columns
API routes	**151**	16 endpoint modules, §6
Engines	**10**	§5
Domain enumerations	56 classes	`app/core/constants.py`
Roles / permissions	6 roles, **20** permissions	`app/core/permissions.py`
Event types	41 defined, **14** reacted to	`EventType`, `sync_engine.react`
Plan operations	**7**	`pending_ops`, §5.1
AI agents / tools	6 agents, **30** tools	§8.3
Background job types	**3**	`process_meeting`, `plan_meeting`, `send_message`
Tests	**335 collected, 335 passing**	49 files, §13 · plus **38** Android JVM tests
Android client	172 files, 19,408 lines	calls 81 of the 151 routes, §7

---

2. Architecture



Startup sequence — `app/main.py:124`

1. Create or migrate the schema. Small idempotent in-place migrations run **before** `create_all`, so a table rename happens while the new name is still free.
2. Seed the platform Super Admin.
3. Register the background job handlers.
4. **Install the Sync Engine as the event-stream reactor.** This single line is what makes the system reactive rather than batch-driven.
5. Start the job workers, first recovering anything a previous crash left running.
6. Start the 60-second scheduler.
7. Warm the dictation speech model in the background — best-effort, so a failure here can never block startup.

3. Multi-tenancy and security

This is the most deliberately engineered part of the system.

Organisation context is server-derived, never client-supplied. `app/api/deps.py` resolves an `OrgContext(org_id, user_id, role, email, name)` from the JWT on every request. No endpoint accepts an `org_id` from a request body, and the AI model is never given one — agent tools receive it by injection (`app/ai/agents/base.py:35`), so a prompt-injected model physically cannot address another tenant's data.

Three isolation layers

1. Application scoping — every query filters on `org_id`.
2. A row-level `org_id` stamped on every business table via `OrgScopedMixin`, so rows stay fenced even without following foreign keys.
3. PostgreSQL Row-Level Security in production (`scripts/postgres_rls.sql`).

Files and AI memory are namespaced too. `storage.resolve()` refuses any path that does not begin `orgs/{caller_org_id}/`, then re-checks the resolved absolute path against the storage root — defeating traversal on both the logical and the physical path. The memory store filters by `org_id` on every search.

The platform operator cannot read customer data. The Super Admin role is explicitly rejected by `get_org_context` and appears in **none** of the org-data permissions in the RBAC matrix (`app/core/permissions.py`). They can create organisations and read platform-level counts; nothing else.

Credentials

Item	Handling
Passwords	PBKDF2-HMAC-SHA256, 390,000 iterations, constant-time comparison
Sessions	JWT HS256, configurable expiry (default 24h), refresh endpoint
Integration secrets	Fernet-encrypted at rest (`app/core/crypto.py`)
Access control	Role permission matrix plus per-record ownership checks

Ownership as well as role. Role alone proved insufficient: all five org roles hold `manage_tasks`, so a role check would not stop one agent modifying another's task. Task-mutation routes therefore enforce ownership through the same matrix used for reassignment — Sales Agent acts on own work, Lead across the team, Admin across the organisation. Completing and reporting progress are deliberately left open, because reporting that work is finished is legitimate teamwork.

---

4. The event stream

`app/engines/events.py` is small and load-bearing. Any meaningful change calls `emit()`, which writes a `progress_events` row and hands it to the Sync Engine inside the same transaction.

The Sync Engine reacts to a subset of event types, and its reactions may emit further events — a recursion-depth guard stops runaway chains, and a failing reaction is logged without losing the event that caused it.

The value of this design: the project timeline screen, the audit trail and the automation triggers are **all the same data**. There is no separate workflow engine that can drift out of step with what actually happened.

---

5. The ten engines

Each engine owns one job, its own triggers and its own tables.

5.1 Planner Engine — `app/engines/planner_engine.py`

Turns meetings and client replies into a versioned action plan.

Entry point. `process_meeting()` branches three ways:

- **New client, concrete discussion** → propose a project with a full task breakdown.
- **Vague discussion** → follow-ups only, no project. This is what stops the system manufacturing projects out of exploratory chats.
- **Existing project** → merge into a new plan version, guarded by a *wrong project* check that holds the merge if the transcript does not appear to concern the project it was filed under.

`create_plan_version()` **is the single mutation path.** A plan is never edited in place. Every change creates a new version carrying an AI-written diff, and:

- supersedes any sibling draft, so nothing half-applies;
- **adopts** tasks a human already confirmed rather than duplicating them — matched by title similarity, with the human's wording and dates winning, and the plan contributing only phase, skill, priority and dependencies;
- wires dependencies by title reference and refuses circular graphs;
- holds the plan as a draft when an unresolved conflict exists, rather than picking a side;
- routes through the Approval Engine, with changes below every rule auto-activating as `"system"`.

`activate_plan()` carries two independent guards against activating a stale version over a newer one — added after a real incident in which approving an old approval card walked a live project backwards.

Also owns: conflict resolution, phase shifts, stakeholder synchronisation from meeting content, and a feasibility pass through the Resource Engine before anything is promised to a client.

5.2 Assignment Engine — `app/engines/assignment_engine.py`

Chooses who does each task, on facts rather than availability alone.

A strict candidate ladder, in order:

1. Project members holding the required skill.
2. Any organisation member holding the skill.
3. The phase owner — explicitly flagged *"no skill match"*, never silently.

Scoring

```
score = skill_level×10 − (open_tasks / capacity)×6 − field_days×0.5 + continuity×1.5
```

Leave, travel bookings, due-date fit and prior work on the same project all feed in. Batch assignment maintains a running load count so the same available expert cannot win every task in one wave.

Behaviour that matters operationally

- Every pick writes a human-readable reason, shown to the assignee — *"Asha: expert skill match, 3/8 open tasks, did 2 related tasks on this project"*.

- Confidence derives from skill match. Below the organisation's threshold the decision routes to a named human instead of being applied.
- A genuine tie is not broken silently — it goes to the lead as a question, and the caveat travels with the reason.
- A high-priority task may exceed someone's capacity, but the engine must then propose what to postpone.
- `decline_task` and `revalidate_assignee` handle a person saying no, and re-check the choice at the moment a blocked task becomes ready.

5.3 Resource & Calendar Engine — `app/engines/resource_engine.py`

The single owner of *who is free when*, across all projects.

Leave, delivery days, demos, travel and provisional holds all become `resource_bookings` rows with a recorded source, so no booking exists without an explanation. It answers three questions: `conflicts_for` (human-readable reasons a person is unavailable), `feasibility` (can these people work this window), and `field_days_in_week` (how much time on the road someone already has).

This is the reason nobody is ever booked in two places at once, and the reason the Assignment Engine's load figures reflect reality rather than task counts alone.

5.4 Sync Engine — `app/engines/sync_engine.py`

The heartbeat: reacts to events, and sweeps daily.

Reactions

Event	Reaction
Task completed	Cancel its reminders, unblock dependents (each re-validated as it becomes ready), close the commitment it fulfilled, and — if client-visible — trigger an update draft
Task blocked	Stop nudging the blocked person and chase the blocker instead
Task unblocked	Restore the work to the ladder
Leave recorded	Redistribute affected work
Deal agreed	Move from selling to delivering: operations co-ownership and the preparation wave
Member left	Redistribute their open tasks
Vendor slip	Escalate and re-date dependent work
Complaints, conflicts, incidents, overdue	Recompute project health

Daily sweep, per organisation

Sweep	Behaviour
Task nudges	Three levels — push, then WhatsApp, then escalation to the lead with a suggested replacement
Commitments	Watches promises in both directions: what we promised and what the client promised
Reply watches	Unanswered outbound → nudge → nudge → create a call task
Overload	Flags people carrying more than their capacity
Revivals	Wakes dormant leads on their revival date with an AI-written context brief
External waits	Chases outside parties whose promises gate internal work

Reminders never fire outside working hours, never on a leave day, and are cancelled the instant the underlying item completes.

Project health (`on_track` / `at_risk` / `delayed`) is recomputed from overdue tasks, blocked tasks, broken commitments, open conflicts and open incidents. The same function returns the ranked *reasons*, so the health badge and the assistant's "what's blocking this?" answer are guaranteed to agree — they read the same source. Health also rolls up from child projects to chain parents.

5.5 Communication Engine — `app/engines/communication_engine.py`

Drafts every outward message. It never sends.

`draft_client_message()` is the single door, and it enforces in order:

1. `do_not_contact` — an absolute stop, checked even when the draft has no project context.
2. The contact has a usable address.
3. The contact is not flagged `needs_review` — nothing is ever sent to an unverified address.
4. A real sender can be resolved, so the client always sees a person.
5. The body is composed from **supplied facts only** — the model is instructed never to invent numbers, dates or promises, and never to assume an industry the facts do not state.
6. A coverage check confirms the draft answers every point the client actually raised.
7. Working-hours scheduling, plus a courtesy cap on how many messages one recipient receives per week.
8. Hand to the Approval Engine.

Every refusal writes a decision row. Two of these paths previously returned silently, meaning a client update could evaporate with no trace anywhere. That class of failure is now structurally closed: stopping is allowed, stopping invisibly is not.

`redraft_rejected()` handles the rejection loop — a rejection note becomes the redraft instruction, and the redraft is given the same rich context as the original draft rather than being handed the old text alone.

5.6 Document Engine — `app/engines/document_engine.py`

Builds the artifacts a project produces, and treats human-made ones as equals.

- `add_deliverable` / `next_version` / `latest_deliverable` — versioned artifacts. A deliverable generated by the system and one uploaded by a person are the same kind of record, and the rest of the machine cannot tell them apart.
- `generate_text_artifact` — drafts an artifact's text with the model when available, and otherwise assembles an honest structured document from the real data it was given. It never invents content to fill a gap.
- `build_amendment` — a scope change produces an amendment that references the agreement it modifies, rather than quietly editing it.
- `coverage_check` — the shared check that a reply addresses every extracted point, used by the Communication Engine.

5.7 Inbound Engine — `app/engines/inbound_engine.py`

Reads every incoming email and WhatsApp message. Nothing is ever dropped.

`ingest()` runs **match** → **understand** → **route**:

- **Match** — sender to contact, then to thread, then to project. Where the thread and the message content disagree about which project is meant, content wins.

- **Understand** — intent, change type, sentiment, a summary, and the specific asks broken into a checklist the coverage check later ticks off.
- **Route** — one of eight intent handlers.

Guarantees

- An unrecognised sender goes to a **triage queue** with a notification to the sales lead — never to the bin.
- Low model confidence marks the message `needs_human` and routes it to the project owner *by name*.
- A message spanning several projects is posted to every affected timeline and put in front of a person.
- **Content overrides surface intent** where the consequences differ. A message that reads conversationally but contains a commercial ask is routed to the machinery that requires approval, not to the casual reply path.
- People a message introduces ("our new coordinator Meena will handle this") are synchronised onto the stakeholder map automatically — humans edit the result rather than data-entering it.

5.8 Approval Engine — `app/engines/approval_engine.py`

Finds the right approver, and records every decision.

Rules are data, not code. `approval_rules` holds a condition (`{"discount_pct_gt": 10}`), a rank, an approver role, a fallback, an optional recommender and an optional joint approver. The condition evaluator is generic — `_gt`, `_gte`, `_lt`, `_lte` and equality — and a missing fact fails the test, so a rule never fires on unknown data. First matching rule by rank wins. Changing who signs off on what is an administrative action, not a release.

Approval shapes supported

Shape	Behaviour
Single	One person decides
Chained	A recommender reviews first; their recommendation travels with the request to the decider
Joint	Two approvers must both agree; either one's refusal ends it
Bundled	Several artifacts are released by one decision and succeed or fail together
Auto-approve	Below-threshold changes clear without a human, still recorded

Pending approvals are reminded at half their expiry window and **escalated to the org admin on expiry** — an approval can never die quietly in someone's inbox.

Approving releases the entity; rejecting **with a note** loops the draft back, and the note becomes the redraft instruction.

5.9 Knowledge Engine — `app/engines/knowledge_engine.py`

The organisation's memory.

- `distil_project` — at closure, the whole engagement is boiled down into durable memories, client intelligence and a cross-sell note.
- `distil_incident` — a resolved problem becomes a lesson, which becomes a **playbook suggestion** an admin can accept with one tap. This is how a mistake made once becomes a standard step thereafter.
- `replay_story` — the narrative history of a client relationship.
- `planner_brief` — rich priors handed to the Planner before it plans: known stakeholders, logistics, risks and lessons already absorbed, so each new plan starts smarter than the last.

5.10 Assistant Engine — `app/engines/assistant_engine.py`

Everything a person reads rather than clicks.

- `build_daily_digest` — a personal morning briefing: today's work, genuinely late work, approvals waiting, overnight client replies, follow-ups due and demos ahead.
- `build_weekly_client_report` / `build_weekly_internal` — the weekly outward report (drafted for approval, never auto-sent) and its internal counterpart.
- `project_brief` and `onboarding_brief` — what a project is, and what a person joining one mid-flight needs to know.
- `whats_blocking` — the ranked answer to "why is this stuck?", sharing its source with the health badge so the two cannot contradict each other.
- `why_trace` — the decision trail behind any record, powering the app's "Why?" chip.

---

6. The API surface

151 routes across 16 modules, all under `/api/v1` except the two health routes. Every path below was enumerated from the running application, and the *Access* column is the actual dependency guarding it — a named permission from the RBAC matrix, `org member` (any signed-in member of an organisation, which also rejects the Super Admin), `super admin`, or `signed in` (authenticated but not yet organisation-scoped, used only by the auth routes).

The **Used by app** column is the Android client's real coverage, taken from its Retrofit interface: **81 of the 151** are called by the app today. The remainder are reachable and tested, but no screen consumes them yet — see §7.

Interactive documentation is served at `/docs` (OpenAPI) when the backend runs.

Authentication (5)

Method	Path	Access	Used by app
POST	`/api/v1/auth/change-password`	signed in	yes
POST	`/api/v1/auth/email-app-password`	signed in	no
POST	`/api/v1/auth/login`	public	yes
GET	`/api/v1/auth/me`	signed in	yes
POST	`/api/v1/auth/refresh`	signed in	yes

Platform (Super Admin) (4)

Method	Path	Access	Used by app
POST	`/api/v1/orgs`	super admin	no
GET	`/api/v1/orgs`	super admin	no
GET	`/api/v1/orgs/platform/stats`	super admin	no
PATCH	`/api/v1/orgs/{org_id}/active`	super admin	no

Organisation administration (20)

Method	Path	Access	Used by app
GET	`/api/v1/admin/audit`	`view_audit`	no
POST	`/api/v1/admin/demo-teams`	`manage_teams`	no

Method	Path	Access	Used by app
GET	/api/v1/admin/demo-teams	org member	yes
PUT	/api/v1/admin/integrations/calendar	manage_settings	no
PUT	/api/v1/admin/integrations/email	manage_settings	no
PUT	/api/v1/admin/integrations/whatsapp	manage_settings	no
POST	/api/v1/admin/message-templates	manage_templates	no
GET	/api/v1/admin/message-templates	org member	no
POST	/api/v1/admin/programs	manage_programs	no
GET	/api/v1/admin/programs	org member	no
POST	/api/v1/admin/programs/{program_id}/pricing	edit_pricing	no
GET	/api/v1/admin/programs/{program_id}/pricing	org member	no
POST	/api/v1/admin/programs/{program_id}/proposal-templates	manage_templates	no
GET	/api/v1/admin/programs/{program_id}/proposal-templates	org member	no
GET	/api/v1/admin/settings	manage_settings	no
PATCH	/api/v1/admin/settings	manage_settings	no
POST	/api/v1/admin/users	manage_staff	no
GET	/api/v1/admin/users	manage_staff	yes
PATCH	/api/v1/admin/users/{user_id}	manage_staff	no
POST	/api/v1/admin/users/{user_id}/reset-password	manage_staff	no

Leads, contacts & consent (10)

Method	Path	Access	Used by app
POST	/api/v1/leads	manage_leads	yes
GET	/api/v1/leads	view_leads	yes
PATCH	/api/v1/leads/contacts/{contact_id}	manage_leads	yes
GET	/api/v1/leads/{lead_id}	view_leads	yes
PATCH	/api/v1/leads/{lead_id}	manage_leads	yes
POST	/api/v1/leads/{lead_id}/consent	record_meeting	yes
POST	/api/v1/leads/{lead_id}/contacts	manage_leads	yes
POST	/api/v1/leads/{lead_id}/contacts/from-card	manage_leads	yes
POST	/api/v1/leads/{lead_id}/intelligence	manage_leads	yes
PATCH	/api/v1/leads/{lead_id}/state	manage_leads	yes

Meetings (7)

Method	Path	Access	Used by app
POST	/api/v1/meetings/notes	record_meeting	yes

Method	Path	Access	Used by app
POST	/api/v1/meetings/schedule	record_meeting	no
POST	/api/v1/meetings/upload	record_meeting	yes
GET	/api/v1/meetings/{meeting_id}	org member	yes
POST	/api/v1/meetings/{meeting_id}/retry	record_meeting	yes
GET	/api/v1/meetings/{meeting_id}/review	org member	yes
POST	/api/v1/meetings/{meeting_id}/review/confirm	record_meeting	yes

Projects (17)

Method	Path	Access	Used by app
GET	/api/v1/projects	org member	yes
GET	/api/v1/projects/resolve/{entity_type}/{entity_id}	view_leads	yes
GET	/api/v1/projects/{project_id}	org member	yes
GET	/api/v1/projects/{project_id}/brief	org member	yes
POST	/api/v1/projects/{project_id}/children	org member	yes
GET	/api/v1/projects/{project_id}/conflicts	org member	yes
POST	/api/v1/projects/{project_id}/conflicts/{conflict_id}/resolve	org member	yes
POST	/api/v1/projects/{project_id}/feasibility	org member	no
POST	/api/v1/projects/{project_id}/members	org member	yes
DELETE	/api/v1/projects/{project_id}/members/{user_id}	org member	yes
POST	/api/v1/projects/{project_id}/phase-shift	org member	yes
GET	/api/v1/projects/{project_id}/plans	org member	yes
POST	/api/v1/projects/{project_id}/stakeholders	org member	yes
POST	/api/v1/projects/{project_id}/status	org member	yes
GET	/api/v1/projects/{project_id}/timeline	org member	yes
GET	/api/v1/projects/{project_id}/weekly-report	org member	yes
GET	/api/v1/projects/{project_id}/whats-blocking	org member	yes

Workflow: tasks, approvals, inbound, incidents (23)

Method	Path	Access	Used by app
GET	/api/v1/approvals	org member	yes
POST	/api/v1/approvals/{approval_id}/decide	org member	yes
POST	/api/v1/assignments/confirm-batch	confirm_assignments	yes
GET	/api/v1/assignments/proposed	org member	yes

Method	Path	Access	Used by app
GET	`/api/v1/digests/daily`	org member	yes
POST	`/api/v1/inbound/ingest`	org member	yes
GET	`/api/v1/inbound/triage`	org member	yes
POST	`/api/v1/inbound/{message_id}/resolve`	org member	yes
GET	`/api/v1/incidents`	org member	no
POST	`/api/v1/incidents/{incident_id}/fix-tasks`	org member	no
POST	`/api/v1/incidents/{incident_id}/resolve`	org member	no
POST	`/api/v1/messages/draft`	org member	yes
POST	`/api/v1/sync/sweep`	org member	no
POST	`/api/v1/tasks/{task_id}/block`	org member	yes
POST	`/api/v1/tasks/{task_id}/complete`	org member	yes
POST	`/api/v1/tasks/{task_id}/decline`	org member	yes
POST	`/api/v1/tasks/{task_id}/new-date`	org member	yes
POST	`/api/v1/tasks/{task_id}/progress`	org member	yes
POST	`/api/v1/tasks/{task_id}/split`	org member	yes
POST	`/api/v1/tasks/{task_id}/unblock`	org member	yes
POST	`/api/v1/tasks/{task_id}/vendor-promise`	org member	yes
POST	`/api/v1/tasks/{task_id}/vendor-slip`	org member	yes
GET	`/api/v1/why/{entity_type}/{entity_id}`	org member	yes

Action items (3)

Method	Path	Access	Used by app
GET	`/api/v1/action-items`	`manage_tasks`	yes
POST	`/api/v1/action-items`	`manage_tasks`	no
PATCH	`/api/v1/action-items/{item_id}`	`manage_tasks`	no

Proposals (7)

Method	Path	Access	Used by app
POST	`/api/v1/proposals`	`send_proposal`	no
GET	`/api/v1/proposals/{proposal_id}`	`view_leads`	yes
POST	`/api/v1/proposals/{proposal_id}/approve-send`	`send_proposal`	yes
GET	`/api/v1/proposals/{proposal_id}/download`	`view_leads`	yes
POST	`/api/v1/proposals/{proposal_id}/edit`	`send_proposal`	yes
POST	`/api/v1/proposals/{proposal_id}/submit-review`	`send_proposal`	yes
POST	`/api/v1/proposals/{proposal_id}/upload-revision`	`send_proposal`	yes

Outbound messages & provider webhooks (6)

Method	Path	Access	Used by app
GET	`/api/v1/messages`	`send_messages`	yes
GET	`/api/v1/messages/{message_id}`	`send_messages`	yes
POST	`/api/v1/messages/{message_id}/approve`	`send_messages`	yes
POST	`/api/v1/messages/{message_id}/reject`	`send_messages`	yes
GET	`/api/v1/webhooks/whatsapp`	public	no
POST	`/api/v1/webhooks/whatsapp`	public	no

Follow-ups (3)

Method	Path	Access	Used by app
GET	`/api/v1/followups`	`view_leads`	yes
POST	`/api/v1/followups`	`manage_leads`	no
POST	`/api/v1/followups/{followup_id}/complete`	`manage_leads`	yes

Demos (4)

Method	Path	Access	Used by app
GET	`/api/v1/demos`	`view_demos`	yes
POST	`/api/v1/demos`	`schedule_demos`	yes
GET	`/api/v1/demos/calendar`	`view_demos`	yes
PATCH	`/api/v1/demos/{demo_id}`	`update_demo_status`	yes

Operations: money, resources, playbooks, rules (19)

Method	Path	Access	Used by app
POST	`/api/v1/approval-rules`	`manage_settings`	no
GET	`/api/v1/approval-rules`	org member	no
DELETE	`/api/v1/approval-rules/{rule_id}`	`manage_settings`	no
POST	`/api/v1/bookings`	org member	no
GET	`/api/v1/bookings`	org member	no
POST	`/api/v1/deliverables`	org member	no
GET	`/api/v1/deliverables`	org member	no
POST	`/api/v1/invoices`	org member	no
GET	`/api/v1/invoices`	org member	no
POST	`/api/v1/invoices/{invoice_id}/record-payment`	org member	no
POST	`/api/v1/invoices/{invoice_id}/send`	org member	no
POST	`/api/v1/leave`	`manage_staff`	no
POST	`/api/v1/playbooks`	`manage_programs`	no
GET	`/api/v1/playbooks/suggestions`	org member	no

Method	Path	Access	Used by app
POST	`/api/v1/playbooks/suggestions/{suggestion_id}/decide`	`manage_programs`	no
POST	`/api/v1/skills`	org member	no
GET	`/api/v1/skills`	org member	no
POST	`/api/v1/vendors`	org member	no
GET	`/api/v1/vendors`	org member	no

Assistant (3)

Method	Path	Access	Used by app
POST	`/api/v1/assistant/command`	org member	yes
GET	`/api/v1/assistant/history`	org member	yes
DELETE	`/api/v1/assistant/history`	org member	yes

Notifications, devices, files, dictation, privacy (7)

Method	Path	Access	Used by app
POST	`/api/v1/devices`	org member	no
POST	`/api/v1/devices/unregister`	org member	no
POST	`/api/v1/dictate`	org member	yes
GET	`/api/v1/files/{rel_path:path}`	org member	yes
GET	`/api/v1/notifications`	org member	yes
POST	`/api/v1/notifications/{notification_id}/read`	org member	yes
POST	`/api/v1/privacy/erase-contact`	`erase_personal_data`	no

Health (2)

Method	Path	Access	Used by app
GET	`/`	public	no
GET	`/health`	public	no

6.1 Conventions that hold across every route

- **Errors.** A typed `AppError` becomes `{"detail": "human message"}` with the matching status: 400 validation, 401 unauthorised, 403 permission denied, 404 not found, 409 conflict, 422 validation failed (also carrying the "needs confirmation" case), 429 budget exceeded, 502 external service.
- **Organisation scope** is derived from the JWT on every request and never read from a body or a query parameter.
- **Idempotency — read this before designing a retry. No route reads the `Idempotency-Key` header.** No handler declares a `Header` parameter and the only middleware is CORS, so the header the Android app stamps on every state-changing call is inert, and `idempotency_records` serves exactly one thing: `jobs/mail_poller.py`, deduping *inbound IMAP* mail by Message-ID. What is real is **server-side keying on message creation**: `message_service.create_message` mints the key itself (`uuid4().hex`, or a deterministic string like `proposal-email-{id}-{addr}`) and returns the existing row on a match, so a retry cannot double-send — while an unapproved draft is refreshed from current content rather than frozen (F61).

Everywhere else, safety comes from a rule specific to the route: a retried `POST /meetings/upload` is refused **422** because the consent record is single-use, and a replayed `POST /approvals/{id}/decide` is refused **409**. A retried `POST /leads`, `/vendors`, `/approval-rules` or `/action-items` **creates a second row**. (Corrected 2026-08-25; the previous wording implied a general guarantee and misled a reviewer.)

- **One clock.** Instants are stored in **UTC** (`models.base.utcnow`). Every question of the form *"which day is that?"* is answered in the **organisation's** timezone (`org_settings.timezone`, default `Asia/Kolkata`) through `app/core/clock.py` — `clock.today(tz)` and `clock.local_date(instant, tz)`, never `date.today()` and never `.date()` on a stored column. Mixing the two is what made a task due 20:00 UTC — 01:30 tomorrow morning in Kolkata, thirty minutes away — read as *overdue*. `tests/test_one_clock_guard.py` AST-bans the bare spellings in `app/`, so a new call site that reaches for the server's own clock fails the suite.
- **Pagination.** List endpoints take `limit`/`offset` and cap `limit` server-side (200 for most lists, 500 for the audit log and timelines).

---

7. The mobile client, and where it diverges

The Android app (`com.yuvipep.app`, Java, MVVM, Hilt) is the only client today. Its Retrofit interface declares **82 methods over 81 distinct routes**, and **every one of them resolves to a route this backend actually serves** — there are no calls to endpoints that do not exist. What follows is the honest inverse: what the backend offers that the app does not yet use, and the two places where the app and the server disagree.

7.1 What the app does not call (70 routes)

Area	Routes	Why it is not a defect
Organisation administration (`api/admin.py`)	25	Staff, programs, pricing, proposal and message templates, settings, the audit log and the three `PUT /admin/integrations/*` endpoints. Configured server-side or by seed script; the in-app Admin tab is an honest placeholder
Operations and money (`api/ops.py`)	20	Invoices (list, create, send, record-payment), vendors, bookings, leave, deliverables, skills and playbooks + suggestions. Backend-complete; the Delivery tab is a placeholder
Platform / Super Admin (`api/organisations.py`)	4	The `platform` product flavour exists but is unimplemented — the `org` build says so rather than pretending
Incidents (`api/workflow.py`)	3	List, add fix-tasks, resolve. The Incidents tab is a placeholder
Approval rules (`api/ops.py`)	3	Rules are data, edited server-side; changing who approves what needs no release
Device push registration (`api/misc.py`)	2	See §7.2 — this one *is* a gap

Area	Routes	Why it is not a defect
Everything else	13	`POST /action-items` + `PATCH /action-items/{id}` (the app uses the richer `/tasks/*` lifecycle routes), `POST /proposals` (see the note below), `POST /meetings/schedule`, `POST /followups`, `POST /projects/{id}/feasibility`, `POST /sync/sweep`, `POST /privacy/erase-contact`, `POST /auth/email-app-password`, both `/webhooks/whatsapp` (called by the provider, not a client), and `GET /` + `GET /health`

Those seven rows sum to **70**, and the split is generated from the code by `scripts/check_docs_in_sync.py` rather than counted by hand.

On `POST /proposals`. The app never calls it, yet the assistant creates proposals perfectly well — both statements are true, and the distinction is worth being precise about. The app's six proposal routes all act on a proposal that already exists (`{id}`: get, edit, download, upload-revision, submit-review, approve-send). Creation is reached by sending a natural-language command to `POST /assistant/command`; the Proposal Agent's `create_proposal` tool then calls `proposal_service.build_proposal` **in-process on the server** — the same function the REST route calls. So the capability is fully wired; only the HTTP creation route is unused by this client.

7.2 Push notifications do not currently reach a phone

This is stated plainly because the backend half looks finished and is not reachable end to end:

- The backend **is** complete: `push_service.py` implements FCM HTTP v1 with a hand-rolled RS256 service-account grant, `device_tokens` stores registrations, dead tokens are pruned, and `POST /devices` accepts them.
- The **app has no Firebase dependency and never calls** `POST /devices`, so no token is ever registered. `NotificationPollWorker` says so in its own comment: the 15-minute poll runs *"until FCM in Phase 8"*.

Delivery today is therefore **polling only**, and it works:

Mechanism	Interval	Behaviour
`ForegroundPoller`	60 s while a screen is open	Refreshes the feed, raises an in-app snackbar for new arrivals
`NotificationPollWorker`	15 min via WorkManager	Fetches the delta and raises *local* system notifications, grouped per channel, deduplicated against the newest stamp already alerted

The design intent holds either way: the `notifications` row is the source of truth and push is only the doorbell. Enabling push needs the Firebase SDK and one call to `POST /devices` on the client; nothing on the server has to change.

7.3 The client's permission mirror — fixed, and now guarded

`RoleGate.java` calls itself a "1:1 mirror of the backend's `app/core/permissions.py`". Until 2026-08-25 it was not: it mapped **19 of the backend's 20** permissions, and the missing one was `confirm_assignments` — added when [F51] made confirming an assignment batch a lead's job. The Home screen offered the assignment-batch card with no role check, so an agent could see it, open the screen, choose assignees and be refused by the server on the last tap. `GET /assignments/proposed` is org-wide with no user filter, so the rows arrive for everybody; only `POST /assignments/confirm-batch` is gated.

Both halves are fixed. `RoleGate` now maps all 20 and `HomeFragment` gates the card on `RoleGate.has(role, CONFIRM_ASSIGNMENTS)`.

The durable part is the guard: `RoleGateTest` diffs the app's matrix against a **generated** `permissions.json` — produced from `_MATRIX` itself, not transcribed — so the next permission added to the backend and forgotten in the app fails an Android build rather than surfacing as a mystery 403 on a phone. A mirror with a hole in it looks exactly like a mirror until someone stands in front of the hole.

7.4 What the client contributes to the contract

Behaviour the backend depends on the app to provide, and which it does:

- **Consent before recording.** The app captures consent through its own sheet and `POST /leads/{id}/consent` *before* uploading; the backend refuses an upload with no prior consent row, so the order is enforced on both sides.
- **Idempotency keys minted at intent time,** so an offline-queued approval decision and its later replay share one key — though **the server does not read them** (see §6.1). What actually makes that replay safe is `decide_approval` refusing a second decision with 409.
- **An offline write queue** that replays `{method, path, body}` in order and surfaces a 4xx refusal with the backend's own message rather than dropping it.
- **A stable assistant** `session_id` persisted across process death, so a conversation can be replayed from `GET /assistant/history`.
- **Cache-first approvals** in Room, so the "waiting on you" feed survives a dead network.

---

8. The AI layer

8.1 Model layer — a vendor switch, not a vendor lock

`app/ai/llm.py` presents one interface over **three providers** (Gemini, Claude, OpenRouter), translating a neutral message format into each vendor's shape, including tool calls and image content. Switching providers is a one-line configuration change. Built in:

- **Two tiers.** A cheap fast model for routing and classification; a strong model for planning. Cost is managed by choosing the tier, not by degrading quality everywhere.
- **Resilience.** A global concurrency cap, exponential backoff with jitter on rate limits and server errors, and **cross-tier fallback** — if the fast model's quota is exhausted it switches to the strong model's separate quota bucket rather than failing the request.
- **Accountability.** Every call records tokens, latency, purpose and cost per organisation, and a per-organisation ceiling is enforced before each call.

8.2 Orchestrator — routing what a person says

`POST /assistant/command` accepts what the user typed or dictated. A cheap routing call picks one of six specialist agents, then that agent runs its own tool loop (bounded at eight rounds).

The router sees the tail of the conversation, so a short follow-up — *"make it warmer"*, *"the second one"*, *"send it"* — routes to the agent that would handle what it refers to, rather than being read as a fresh request. Routing failure is never fatal: it defaults to the read-only Query Agent.

8.3 The six agents

Each agent is one focused worker with a small tool set. The model chooses *which* tool and with what visible arguments; identity and organisation are injected by the runtime and are not model-controllable.

Query Agent — questions and reports

"Answer by calling tools, then present a short numbered list. Never make up data."

Tool	Purpose
`list_action_items`	Tasks by owner, status and client
`list_leads`	Clients by state, search, importance or ownership
`get_client_details`	One client in full: state, contacts, notes, proposals
`get_project_brief`	Objective, status, health and *why*, live plan, task counts, client's people, open promises, what is due next
`list_approvals`	What is waiting for a human decision
`list_followups_due`	Due today, overdue, this week or all pending
`list_demos`	Scheduled demos by month or team
`search_past_notes`	Semantic search over meeting notes and client intelligence
`explain_decision`	The recorded reasoning behind an AI decision

Three prompt rules exist because each was observed failing in the field:

- **"Who is doing X" means which of *our* people.** Asked who was handling a deliverable, the agent once answered with the *client's* coordinator — which inverts responsibility, because the representative then waits on a client who was never asked. It now answers from the task assignee, and if nobody on our side is assigned it must say so explicitly.
- **Assignment reasons are quoted verbatim,** including caveats such as *"tie; lead should confirm"* — that caveat is usually the point of the question.
- **An empty reason is reported as empty,** never backfilled with a plausible one.

Conversation Agent — notes and task changes

"When the user refers to a task by number, fetch the current numbered list first, then act on the matching id."

Tool	Purpose
`add_client_note`	Save client intelligence (also written to long-term memory)
`list_my_action_items`	The caller's tasks, numbered
`create_action_item`	New task, optionally linked to a client
`reassign_action_item`	Move a task to another person, who is notified
`close_action_item`	Mark done with the result note

Proposal Agent — building and editing proposals

"Quantities and prices must come from the user's words or the pricing table — never invent numbers. When asked to create and send, do both steps."

Tool	Purpose
`create_proposal`	Build a draft from a program's template and current pricing
`get_proposal`	Fetch line item codes, quantities and status — called first before any edit
`update_line_items`	Apply commercial edits the user actually asked for
`edit_narrative`	Rewrite the wording, with numbers locked
`prepare_email`	Draft the covering email with the document attached, for approval

The commercial block sits between markers in the document, so a narrative rewrite cannot touch the figures — the computed block is always re-inserted. `prepare_email` resolves the recipient from a name *or a job title in the client's own words*.

Email Agent — outward messages

"Always from the salesperson's own address, never a robot address. Every message waits for one-tap approval; say so in your reply."

Tool	Purpose
`draft_intro_message`	The organisation's templated introduction for a new contact
`draft_custom_email`	A custom or reply email about a client

Follow-up Agent — keeping the loop turning

"Draft short polite messages that mention the last interaction."

Tool	Purpose
`list_due_followups`	What is due or overdue
`draft_followup_message`	Email or WhatsApp follow-up
`schedule_followup`	Create a reminder for a date
`complete_followup`	Mark done — the next reminder is scheduled automatically

Scheduling Agent — demos and teams

"When booking, check the team's availability first and mention any clash you find."

Tool	Purpose
`list_demos`	Scheduled demos with client, date and team
`list_demo_teams`	Teams and their members
`calendar_view`	A month grid, so free days are obvious
`book_demo`	Schedule for a client, optionally with team and time
`cancel_demo`	Cancel a scheduled demo

8.4 Safeguards shared by every agent

Implemented once in `app/ai/agents/base.py`, so a tool added later inherits them without knowing the rules exist.

- **The system prompt always states today's date.** A model with no clock does not say "I don't know the date" — it borrows a weekday from the question and states it as fact. It is also told to report a due date exactly as the tool returned it, and to say when one is not set rather than describing an unset date.
- **The selected client is passed as a name, not an id,** so the model's description of what it is scoped to is accurate, while the tools scope themselves in code.
- **Ambiguity becomes a question, not a guess.** Entity resolvers raise a structured "needs confirmation" carrying candidate options. The model is instructed to ask and stop, and the response explicitly states that nothing was drafted or sent — so a clarifying question can never be mistaken for a completed action.
- **Approval cards tell the truth about the draft behind them.** The preview is long enough to show the whole message, flagged when truncated, and any unfinished blanks are surfaced *before* the tap rather than as a refusal after it.
- **Conversation memory holds plain text only,** scoped to one person, in one organisation, in one session. Tool results and entity ids are never replayed, so every action is freshly authorised against live permissions rather than yesterday's.

8.5 Planning discipline

`app/ai/planning.py` never trusts model output. Responses are schema-constrained, then **re-validated field by field** against allowed vocabularies — phases, priorities, skills, roles — with anything invented outside them dropped or normalised. Truncated output gets a repair pass; the reasoning budget is capped so a long think cannot starve the answer out of the token budget. If planning genuinely fails, the work is **held for a human** — it never emits a half-plan.

8.6 Speech, vision and memory

- **Speech.** Local Whisper by default: audio never leaves the server and there is no per-minute API cost. Field dictation deliberately uses a smaller, faster model and **never persists the audio** — a dictated word is a keystroke, not a record, and keeping them would create a store of employee voice clips nobody asked for.
- **Vision.** Business-card capture, with low-confidence results *flagged for review* rather than silently accepted.
- **Memory.** Per-organisation semantic search over meetings and notes. The embedding model runs **locally** (`EMBEDDING_PROVIDER=local`), so notes never leave the server and there is no per-call cost or quota; `gemini` and a deterministic offline `hash` embedder remain one-line alternatives. The offline embedder is selected by the switch, **not** by a missing key — an earlier version fell back only when the key was absent, so a key that was present but out of credits raised instead and notes went silently unindexed.

8.7 The system runs without AI at all

`app/engines/nlu.py` provides two paths behind one contract: a model path when a key is configured, and a **fully deterministic** keyword-and-grammar path otherwise.

This is why the product is demonstrable, testable and reproducible with no API key and no network — and why a model outage degrades the product rather than stopping it.

9. Background processing

Job queue — database-backed, so a restart loses nothing and interrupted jobs are recovered at boot. Three priority levels keep interactive work ahead of heavy background work, and **per-organisation fairness** prevents one tenant starving another. Failures back off exponentially and eventually dead-letter, and a dead-lettered send *flags the affected message* rather than disappearing. The record contract is deliberately shaped so the executor could be swapped for a distributed queue later without touching callers.

Meeting pipeline — `uploaded → transcribing → transcribed → extracting → review_pending → completed`, polled by the app so nobody stares at a frozen screen. Idempotent: a retry reuses the existing transcript rather than transcribing again.

Scheduler — a 60-second tick. Daily it runs follow-up reminders, the per-org Sync sweeps (one organisation's failure never blocks another's), personal digests, weekly reports and the retention sweep. Every tick it recovers orphaned sends. Optional inbound mail polling runs on its own interval.

Send worker — sliding-window pacing per organisation and channel, idempotent on retry, with provider failures backed off rather than lost.

---

10. Integrations

Integration	Notes
Email (SMTP)	Three-step sender resolution — the user's own address, then the organisation's, then a platform default — so the client always sees a real person
WhatsApp	Meta Cloud API, with delivery receipts arriving by webhook into the delivery log
Calendar	Google Calendar as a *mirror only*; the internal calendar is always the source of truth and Google failures never block
Calendar invites	Standards-compliant invites over plain SMTP, producing native RSVP cards in Gmail and Outlook, with correct handling of updates and cancellations — no Google Workspace or domain-wide delegation required
Push	Firebase Cloud Messaging (HTTP v1), dormant until a service-account key is configured. The in-app notification row is always authoritative and push is only the doorbell. **Backend-complete but not reachable end to end today: the Android client registers no device token — see §7.2**
Documents	Editable and shareable document rendering for proposals and reports

Every integration has a **dry-run mode**, so the entire product can be demonstrated end to end with no external accounts configured.

---

11. Governance and audit

Four independent record trails

Trail	Answers
`audit_log`	Who did what, and when
`progress_events`	What happened in the business, in order
`ai_decisions`	What the AI decided, why, how confident, and whether it deferred to a human
`llm_usage`	What every model call cost, and for what purpose

`GET /why/{entity_type}/{entity_id}` returns the decision trace for any record.

The unfinished-draft fence. Models sometimes write a fill-in-the-blank — `[Name]`, `[Insert Date]` — instead of admitting they lack a fact. This is caught by a **deterministic guard** (`app/services/draft_check.py`), not a prompt; prompts had already been observed failing at exactly this. It checks the subject, the body **and attachment text**, on **both** approval routes, and refuses with the offending lines quoted back. It is not a veto — a human may re-approve explicitly — but it cannot happen by accident. The check is deliberately narrow so it does not block legitimate sends.

Data protection

- Recording consent must exist **before** an upload and is *linked*, never manufactured. An upload with no prior consent record is refused — closing the gap where a self-asserted flag generated its own evidence.
- Retention sweeps delete raw audio and scrub transcripts on per-organisation schedules.
- Assistant chat history is dropped after a configurable window, so a working buffer does not silently become a permanent store.
- Right to erasure removes a person from the database, the files **and** the AI memory store.

---

12. Domain model

Layer	Tables
Tenancy & people	`organisations`, `org_settings`, `users`, `user_skills`, `device_tokens`, `notifications`
Sales	`leads`, `lead_state_history`, `contacts`, `consent_records`, `client_intelligence`, `meetings`, `transcripts`, `followups`, `demos`, `demo_teams`, `demo_team_members`
Delivery	`projects`, `project_members`, `stakeholders`, `action_plans`, `action_items`, `client_commitments`, `conflicts`, `playbooks`, `playbook_suggestions`, `resource_bookings`
Commercial	`programs`, `pricing_tables`, `proposal_templates`, `message_templates`, `proposals`, `proposal_revisions`, `deliverables`, `invoices`, `payments`, `vendors`
Communication	`threads`, `inbound_messages`, `outbound_messages`, `message_delivery_log`, `approvals`, `approval_rules`
Machinery & governance	`progress_events`, `assignments`, `reminders`, `incidents`, `ai_decisions`, `digests`, `jobs`, `llm_usage`, `audit_log`, `idempotency_records`, `memory_chunks`, `assistant_turns`

Two modelling decisions worth attention

`org_settings` **is the product's configurability.** Rather than hard-coding company policy, the behavioural dials live as data per tenant: working days and hours, the escalation ladder gaps, how many messages one client contact may receive per week, the reply-nudge ladder, the AI confidence threshold, and whether assignments apply automatically or need lead confirmation. Onboarding a client with different working practices is configuration, not code.

Nothing is destroyed. Plans supersede rather than overwrite; tasks move to done, cancelled or superseded; proposals accumulate revisions; every state change writes a history row. This is what makes retrospective audit possible at all.

---

13. Testing

335 tests collected, 335 passing across **49 test files**, in about 35 seconds (measured 2026-08-25; was 272 across 38 files). The suite runs against an isolated temporary database with an intentionally invalid API key, so tests can never reach a live model provider or send a real message.

The Android app has its own suite now, and it runs headlessly — no emulator, no device:

```
cd <AndroidStudioProjects>/YuvipepSalesforceAI
./gradlew testOrgDebugUnitTest --offline      # 38 tests
```

Suite	Covers
`ContractTest` (19)	Every DTO the app reads, parsed against **real backend JSON** captured from a live seeded server into `src/test/resources/contract/`. Gson leaves an unknown field null rather than complaining, so a backend rename reaches a user as a blank screen; this turns it into a failing build. Re-capture with `scripts/capture_android_contract.py`.
`RoleGateTest` (5)	The app's permission matrix against a **generated** `permissions.json` (§7.3).
`WorkDayTest` (13)	Date bucketing in the organisation's timezone (§6.1, "One clock").

Two things it deliberately does not cover: `src/androidTest` is **empty**, so fragments, the recorder, runtime permissions and WorkManager scheduling are still verified by hand on a device; and every list fixture is required to hold at least one row, because an empty array parses under any shape and would prove nothing.

```
$ python -m pytest -q
335 passed in 37.66s
```

A time-of-day caveat. 3 of those tests, in `tests/test_what_is_late.py`, carry a test-side timezone assumption rather than a product defect: `assistant_engine.py` partitions "late" on `today_local = date.today()` (the local date), while the test helper builds due dates from `utcnow()` (the UTC date). On a UTC+5:30 machine those two dates disagree between 00:00 and 05:30 local time, so this suite is **green for 18.5 hours a day and red for 5.5** — collected stays 272 throughout; only the passing count drops to 269 inside that window. The fix is one line in the test helper and is outside this project's closed change list.

What is notable is *what* is tested. Rather than only unit-testing functions, the suite locks down behaviours that previously failed in real use — the file names read as a defect register:

`test_silent_refusals` · `test_draft_blanks` · `test_plan_lineage` · `test_task_ownership` · `test_what_is_late` · `test_whats_blocking` · `test_assistant_knows_the_date` · `test_redraft_has_context` · `test_genericity` · `test_inbound_triage` · `test_message_approvals` · `test_silent_refusals` · `test_demo_reschedule_from_meeting` · `test_value_estimate_quantities` · `test_proposal_context_summary` · `test_commitment_register`

13.1 Two standing guards against domain leakage

The product must not assume an industry. Two guards enforce that on every run, because a one-off sweep is a claim made once while a guard is a claim re-made every time:

Guard	Covers	Status
`pytest tests/test_genericity.py`	AST-parses every prompt in `app/` for domain words	**8 passed**
`python scripts/check_hardcoded_strings.py`	The Java that Android Lint's XML-only `HardcodedText` cannot see: user-visible literals, domain words and currency symbols in code	**0 findings** (2026-08-25)

Guard	Covers	Status
`pytest tests/test_one_clock_guard.py`	AST-bans bare `.today()` / `.now()` / `.utcnow()` in `app/`, so every date judgement goes through the organisation's clock	**2 passed**

The Java guard reported 2 findings until 2026-08-25 — cosmetic developer affordances rather than domain leakage: `LoginFragment.java:101` set the literal `"server: "` on the debug environment chip and `SplashFragment.java:48` prefixed the version with `"v"`. Both now read through `getString(R.string.…)`, so it exits 0. Its precision check passes 11/11, and it reports no domain words and no hardcoded currency symbols anywhere.

> ■ ***A green result from the Java guard is machine-dependent.*** *Its default path is the Android project in* `AndroidStudioProjects/`*,* ***outside this repository*** *— there are no* `.java` *files under* `Complete_Project` *and git tracks none. On CI, or any machine without the Android checkout, it prints* `[SKIP] no Java sources` *and exits 0* ***having scanned nothing****. Pass* `--app-dir` *explicitly if you need it to mean something.*

Alongside these sits a scripted verification layer: a deterministic dry-run walkthrough of the workflow catalogue (`scripts/dryrun_*.py`), and **19** `verify_live_*` **scripts** that drive the same journeys against a real model provider.

A signal for technical reviewers. The codebase carries a numbered findings trail cross-referenced to a detailed findings log. Each defensive guard cites the real incident that justified it. This is unusually disciplined: the *reasons* for defensive code sit next to the code, so a future maintainer does not remove a fence without understanding what it holds back.

---

14. Configuration and deployment

Single-file configuration (`.env`) covering application and session settings, database, LLM provider and model tiers, speech provider and model sizes, integration credentials and modes, worker counts and retention windows.

Anything not configured **degrades gracefully into dry-run** rather than failing — the backend always starts and always runs end to end.

```
pip install -r requirements.txt
python scripts/seed_demo.py     # demo organisation, staff, programs, templates
python run.py                   # http://localhost:8000/docs
```

The configuration in force right now, for anyone reading this against the running system. Note that email and calendar are **live**, not dry-run:

Setting	Value	Consequence
`PORT`	`8000`	`http://localhost:8000/docs`
`DATABASE_URL`	SQLite at `./data/salesplatform.db`	No Row-Level Security; app-level scoping is the fence
`LLM_PROVIDER`	`openrouter`	fast `openai/gpt-5.4-nano`, strong `google/gemini-2.5-flash`
`STT_PROVIDER`	`whisper_local`	`medium` for meetings, `tiny` for field dictation; audio stays on the server
`EMAIL_MODE`	`smtp` **— LIVE**	An approved message really is delivered
`CALENDAR_MODE`	`google` **— LIVE**	Demo events really are mirrored

Setting	Value	Consequence
`WHATSAPP_MODE`	`dry_run`	Messages are written to `outbox/`, not sent
`INBOUND_EMAIL_MODE`	`off`	The IMAP poller does not run; `POST /inbound/ingest` still works
`FCM_SERVICE_ACCOUNT_JSON`	unset	Push disabled (and see §7.2)

Production checklist — currently open

Item	Status	Action
`SECRET_KEY`	Placeholder value in the live `.env`	Move to a secrets manager. It derives the encryption key, so rotating it invalidates stored integration secrets, which must be re-entered.
SMTP credentials	A real app password is present in `.env`	Rotate and move to secret storage.
CORS	Wildcard origins with credentials enabled (`app/main.py:156`)	Restrict to known client origins.
Database	SQLite	Point at PostgreSQL and apply `scripts/postgres_rls.sql`.
Schema migrations	`create_all` plus light in-place migrations	Introduce Alembic before the first production schema change.
AI spend ceiling	Set very low	Raise to a realistic figure, or model calls will begin being refused under load.

---

15. Scaling path

Each concern is isolated behind one module, so growth does not require a rewrite.

Pressure	Response	Blast radius
Data volume	SQLite → PostgreSQL with row-level security	Configuration plus one SQL script
Job throughput	In-process queue → distributed queue	The queue module only; the record contract already fits
Semantic search scale	JSON embeddings → a vector database	The memory module only
File volume	Local filesystem → object storage	The storage module only; paths already map one-to-one to object keys
Model cost or quality	Any provider change	Configuration — three vendors are already abstracted

---

16. The principles the code actually enforces

Each of these is verifiable in the source, not an aspiration.

1. **Nothing reaches a client without a human tap.** Every draft enters a pending state; unfinished drafts are refused; every send is idempotent.
2. **The AI never authors commercial figures.** They are computed from versioned organisational data or a recorded human decision. The model writes the words around them.
3. **When unsure, ask a named human.** Confidence is compared to a per-organisation threshold and routed, not rounded up.
4. **Every decision is explainable.** Four independent trails and a "why" endpoint for any record.
5. **Refusing is allowed; refusing silently is not.** Every path where an engine declines to act records the reason and the remedy.
6. **People can always do the work themselves.** Typed notes, manual uploads and hand-created tasks are first-class — the engines reconcile manual work rather than fighting it.

Compiled from direct source review of `app/`, with the test suite executed for verification.